Assignment 2.2 - Improving Version Control Practices and Early Workflow Automation

Date: August 22, 2026

Student: Jonathan Cant

Instructor: Prof. Richard Krasso

NorthRiver Software is a small development team working on a customer-facing web application. They are experiencing several issues:

- Branches often stay open for weeks
- Pull requests are large and difficult to review
- Reviews are inconsistent—some are detailed, others are rushed
- There is no automation around pull requests or reviews
- Developers frequently merge code that breaks existing functionality
- The team has no clear branching strategy

Leadership wants to improve predictability and reduce the amount of time spent fixing issues caused by inconsistent practices.

Required Action:
Part I:
Using the readings from this week, write a 2-3 page analysis (roughly 600-900 words) that addresses the following:

1). Identify Version Control Issues
Based on the scenario, describe the problems you see and why they matter.

2). Recommend a Branching Strategy
Choose a strategy from the readings and explain why it fits this team.

3). Improve Pull Request and Review Practices
Describe what changes you would make to improve consistency and reduce review delays.

4). Introduce Early Workflow Automation
Identify at least two automations that would help this team maintain Flow and Feedback.

5). Connect to the Three Ways
Explain how your recommendations support Flow, Feedback, and Continuous Learning.

How this prepares you for the Midterm (Week 5):
The midterm requires you to evaluate a real development pipeline. The work you're doing here—identifying issues, recommending improvements, and grounding your reasoning in DevOps principles—is the same type of analysis you'll apply in Week 5.

Part I: NorthRiver Software Analysis

1. Identify Version Control Issues

NorthRiver Software has several connected version control problems. Branches staying open for weeks allow them to drift away from the main codebase, which increases merge conflicts and rework. Large pull requests create similar problems because reviewers must process too much information at once, increasing cognitive load and making it easier to miss defects (Krasso, 2026).

I would also look upstream at how work is being scoped. In my experience, the size of a branch and pull request often reflects the size of the user story or bug assigned to the developer. Business Analysts and Product Owners should break larger requirements into smaller, focused tickets and group related work under an Epic when necessary. Estimation practices such as Planning Poker can also expose complexity or differences in understanding before development begins.

The inconsistent review process makes these problems worse. Some changes receive detailed reviews while others are rushed, and developers are already merging code that breaks existing functionality. That shows NorthRiver lacks both predictable Flow and timely Feedback.

2. Recommend a Branching Strategy

For NorthRiver, I would recommend short-lived feature branches. Each feature, bug fix, or improvement should have its own branch tied to a small, clearly defined unit of work. The Week 2 reading notes that feature branching works well when work is broken into manageable pieces, reviews happen quickly, and developers integrate frequently (Krasso, 2026).

I would not move NorthRiver directly to trunk-based development yet. The reading identifies fast, reliable automated tests as an important condition for trunk-based development, and NorthRiver does not currently have that foundation. Short-lived feature branches provide some isolation while the team improves testing and review discipline.

The important point is that branches should remain short-lived. Smaller tickets should naturally produce smaller branches, smaller PRs, faster reviews, and fewer difficult merges.

3. Improve Pull Request and Review Practices

NorthRiver should establish a consistent definition of what a pull request needs before it can be merged. PRs should be small, clearly describe their purpose, reference the associated ticket, and explain how the change was tested. At least one other developer should review and approve the change before it reaches the main branch.

Reviews should focus on correctness, clarity, maintainability, and whether the implementation satisfies the requirement. Formatting and other mechanical checks should be automated rather than consuming reviewer time. Large PRs encourage reviewers to skim, while smaller PRs make meaningful review easier (Krasso, 2026).

I would also avoid adding unnecessary ceremony. Quality gates should reduce meaningful risk without becoming another bottleneck.

4. Introduce Early Workflow Automation

The first automation I would introduce is a GitHub Actions pull-request workflow that runs on every PR. For a web application, that workflow could use GitHub's standard 'actions/checkout' and 'actions/setup-node' actions to prepare the repository and Node.js environment, followed by commands that install dependencies, run linting, execute automated tests, and verify that the application builds successfully. GitHub's 'setup-node' action is specifically designed to configure the Node.js environment used by workflow jobs.

Those jobs could then be configured as required status checks for the main branch. If linting, tests, or the build fails, GitHub would prevent the PR from being merged until the problem is corrected. I would combine those checks with branch protection requiring at least one approved review.

A second useful gate would be GitHub's "Dependency Review Action". It evaluates dependency changes introduced by a pull request and can fail the workflow when newly introduced packages contain known vulnerabilities. When configured as a required check, that failure can block the merge.

I would start with these practical checks before adding more complex automation. AI-assisted review could later summarize PRs or identify risky areas, but it should support the reviewer rather than compensate for an inconsistent workflow.

5. Connect to the Three Ways

These recommendations support all Three Ways. Flow improves because smaller tickets, short-lived branches, and smaller PRs allow changes to move through development more predictably. Feedback improves because automated linting, builds, tests, dependency checks, and human reviews tell developers about problems before code reaches the main branch.

Continuous Learning develops when the team looks for patterns in failed checks, review comments, merge conflicts, defects, and oversized stories. Those patterns can be discussed during planning and retrospectives and used to improve how work is scoped and developed.

I would not try to solve NorthRiver's problems by immediately creating a highly complex DevOps pipeline. I would first improve work sizing, keep branches and PRs small, establish consistent review expectations, and automate the checks developers should not have to remember manually. That creates the predictable foundation NorthRiver needs for more advanced CI/CD and AI-assisted practices.

References:

GitHub. “Configuring the dependency review action”. GitHub Docs. https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/manage-your-dependency-security/configure-dependency-review-action

GitHub. “Setup Node.js environment”. GitHub Marketplace. https://github.com/marketplace/actions/setup-node-js-environment

Krasso, R. (2026). WEB 430 Week 2 readings. Bellevue University.

Talreja, Abhay. Teaching Agile. (Jan 25, 2026). *Planning Poker: The complete guide to Agile estimation for Scrum teams*. Teaching Agile. https://teachingagile.com/scrum/psm-1/scrum-planning-estimation/estimation-techniques/planning-poker

Part II
For this part of the assignment, you will work with a small code snippet and use an AI-assisted tool to generate:

- A **pull request summary**, and
- **Code review comments**

*This gives you practical experience with AI-assisted development, which you will build on in the **Final Project (Week 9)**.*

Code Snippet (JavaScript):

```
function calculateDiscount(price, percentage) {
  if (percentage < 0 || percentage > 100) {
    return null;
  }

  const discount = price * (percentage / 100);

  return price - discount;
}
```

Steps:

1. Copy the code snippet into an AI-assisted tool.
2. Ask the tool to generate a **pull request summary** for this change.
3. Ask the tool to generate **code review comments** for the same snippet.
4. Review the output and reflect on what you notice.

What to Submit for the Hands-on Portion:

1. The AI-generated pull request summary
2. The AI-generated code review comments
3. A short reflection (8-10 sentences) addressing:
4. What the AI did well
5. What it missed
6. Whether the summary or comments would help a real team
7. How this connects to the idea of AI-assisted development

In the final project, you will design an AI-augmented workflow. Understanding how AI can support version control and reviews is an important part of that design.

Part II – AI-Assisted Pull Request Review Using OpenAI GPT-5.4 Nano

AI-Generated Pull Request Summary

What the function does

calculateDiscount(price, percentage) computes the discounted final price after applying a percentage-based discount.

Input validation

The function validates only the percentage value.

It returns null when percentage is outside the range of 0 to 100.

It does not validate price for type, finiteness, or negative values.

Result returned

If percentage is invalid, the function returns null.

Otherwise, it calculates:

discount = price * (percentage / 100)

and returns:

price - discount

This represents the discounted price.

AI-Generated Code Review Comments

Correctness

What should happen if price is null, undefined, non-numeric, NaN, or Infinity?

If price is negative, the function will return a value that may not make sense. Should negative prices be allowed or rejected?

Input Validation

Consider validating price similarly to percentage, including checking that it is a number and a finite value.

Consider whether percentage should accept numeric strings such as "10" or require strict numeric input.

Edge Cases

Floating-point calculations may produce rounding artifacts for some inputs. The function may need to round to a fixed number of decimal places depending on the requirements.

Confirm expected behavior for the boundary values:

percentage = 0, which should return the original price.

percentage = 100, which should return 0.

Maintainability and Clarity

Returning null introduces a number | null return type. The expected behavior for invalid input should be documented.

Consider adding documentation describing the expected input types and the meaning of a null return value.

Question Before Merge

What is the intended behavior when price is invalid? Should the function return null, throw an error, or attempt to coerce the value?

Reflection

I thought GPT-5.4 Nano did a good job of summarizing the purpose of the function and identifying the validation that was already present. More importantly, it did not simply accept the code as correct and move on. It identified several issues that would be worth discussing during a real code review, especially the lack of validation for price, handling of negative values, and JavaScript type coercion. I also thought the comments about boundary values and floating-point precision were useful because those are the kinds of edge cases that should be considered when testing financial calculations.

One limitation is that the AI could not determine the actual business rules for the function. For example, it could identify that negative prices or null return values might be questionable, but it could not decide whether those behaviors were acceptable without additional requirements or business context. I think the pull request summary would help a real development team by giving reviewers a quick understanding of what changed before they begin examining the code.

The review comments would also provide a useful first pass by identifying areas that may deserve closer attention. However, I would still expect the developer and reviewer to decide which recommendations are valid for the application rather than automatically accepting every AI suggestion. This connects directly to AI-assisted development because the AI can reduce some of the cognitive work involved in reviewing code while human engineers remain responsible for understanding the requirements, evaluating the recommendations, and making the final decision.